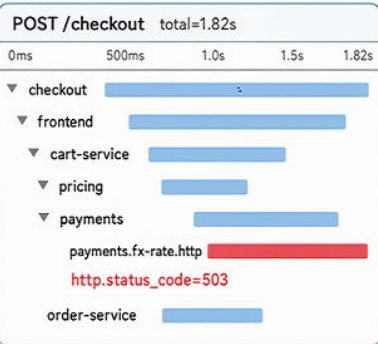


DAV: Data-analyst / SRE

step1 Open Jaeger trace:
Spot one red leaf span



step2 curl + jq pull the span JSON for evidence

```
$ curl -s localhost:16685/
api/traces/$TID | jq
'.data[0].spans[].operationName'
```

payments.fx-rate.http

step3 kubectl logs
confirm upstream timeout pattern

```
$ kubectl edit deploy/fx-rate
--- a/deploy/fx-rate.yaml
+++ b/deploy/fx-rate.yaml
@@ spec.template.spec.containers[0] @@
- timeoutSeconds: 2
+ timeoutSeconds: 5
+ retries: 3
+ readinessProbe.timeoutSeconds: 5

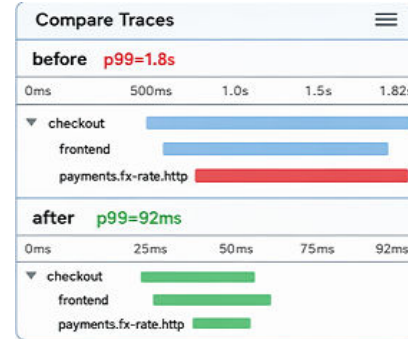
rollout status: successful
```

step4 kubectl edit
deploy: bump timeout, add retry

```
$ kubectl -n payments logs
deploy/fx-rate --since=10m |
grep -i timeout

fx-rate: dial tcp 10.0.4.7:8443
i/o timeout
```

step5 Compare Traces
in Jaeger: p99
1.8s
->92ms



Only CLI:
70% root span is
vision/proportion signal
Only GUI:
JaegerUI can't read
"curl" ".md"

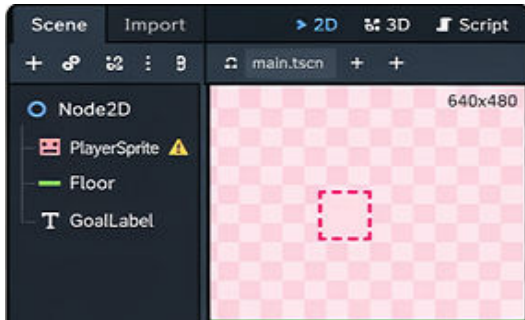
CLI log
verification/revision +
GUI span time analysis

CLI
Operation

GUI
Operation

GAME: Gamedev / QA

step1 Play scene: missing
sprite, flat floor,
label off-screen



step2 Enumerate bugs
from .tscn into
bug list

```
$ python3 tools/scan_scene.py
scenes/main.tscn > bugs.json

3 issues:
missing_texture,
degenerate_extents,
label_offscreen
```

step3 Patch the 3
nodes in
source files

```
$ sed -i 's|res://missing.png|
res://assets/player.png|'
scenes/main.tscn

$ python3 fix_floor_extents.py
scenes/main.tscn # extents=
Vector2(320,8)

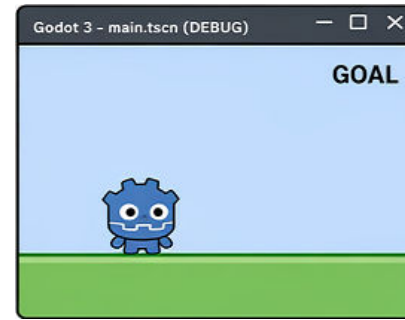
$ python3 fix_label_xy.py
scenes/main.tscn # clamp to
640x480
```

step4 Headless render
one frame as
snapshot.png

```
$ godot3 --no-window --script
tools/snap.gd

wrote snapshot.png (640x480)
```

step5 Replay frame:
sprite, solid floor,
GOAL in-frame



Only CLI:
Can't observe
rendered gameplay
Only GUI:
Bugs are in different
nodes

CLI Bugs location and
Fix + GUI Games Env
Render

CLI
Operation

GUI
Operation

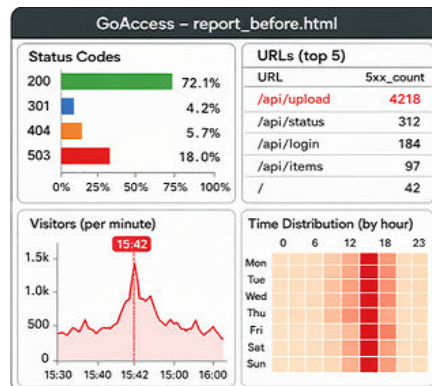
OPS: Web Ops

step1 Baseline load-
test + parse logs
into report.html

```
$ ab -n 5000 -c 50
http://localhost/api/upload >
http_codes_before.txt

$ goaccess access.log -o
report_before.html --log-
format=COMBINED
```

step2 Dashboard:503
spike on /api/upload
at 15:42



step3 Edit nginx.conf:
body size, time
out, rate-limit

```
$ vim /etc/nginx/nginx.conf

# client_max_body_size 50m;
# proxy_read_timeout 60s;
# limit_req_zone
$binary_remote_addr ...

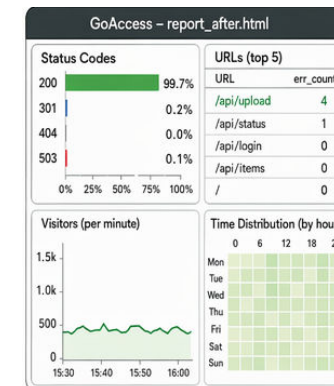
$ nginx -t && systemctl
reload nginx
```

step4 Retest +
regenerate the
dashboard

```
$ ab -n 5000 -c 50
http://localhost/api/upload >
http_codes_after.txt

$ goaccess access.log -o
report_after.html --log-
format=COMBINED
```

step5 Dashboard goes
flat:
503 38% -> 0.1%



Only CLI:
URL*Time*State*
Concurrency
Fourth Power Complexity
Only GUI:
Unable to Write in -
Execution - Verify

CLI Fast Modify +
GUI Four States
Observation

CLI
Operation

GUI
Operation